

Introduction to TypeScript

TypeScript is a powerful superset of JavaScript developed by Microsoft. It adds static typing to enhance code quality and maintainability.

First released in 2012, TypeScript has become an essential tool for modern web development.

 by Saratha Natarajan



Why Use TypeScript?



Early Error Detection

Catches errors during development rather than runtime.



Code Quality

Improves maintainability with clear type definitions.



Developer Experience

Enables powerful IDE support with intelligent autocomplete.



TypeScript vs JavaScript

TypeScript

- Static typing
- Compile-time checks
- Enhanced tooling
- Additional syntax

JavaScript

- Dynamic typing
- Runtime checks
- Wider compatibility
- Simpler syntax

Setting Up TypeScript

Install Prerequisites

Download and install Node.js and npm from nodejs.org.

Install TypeScript

Run

```
npm install -g typescript
```

in your terminal.

Create TypeScript Files

Create files with the .ts extension for TypeScript code.





Compiling TypeScript Code



Create .ts File

Write TypeScript code in app.ts



Run Compiler

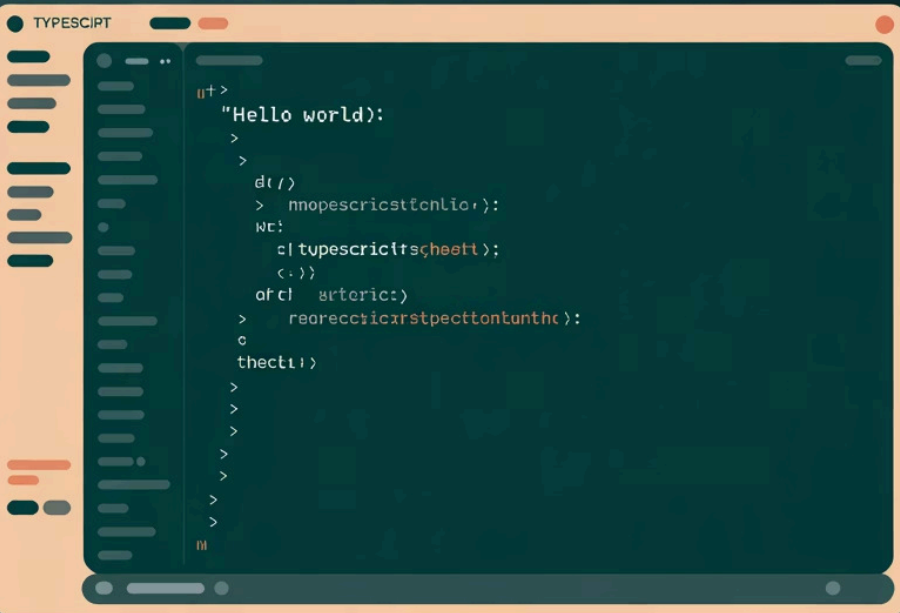
Execute `tsc app.ts` in terminal



JavaScript Output

Compiler generates app.js file

Hello World in TypeScript



Code Example

```
let message: string =  
"Hello World";
```

```
console.log(message);
```

Type Annotation

The **: string** syntax tells TypeScript this variable must be a string.

Execution

After compilation, run with Node.js to see "Hello World" in the terminal.

Variables and Data Types

A

String

Text values:

```
let name: string  
= "John";
```



Number

Numeric values:

```
let age: number  
= 30;
```



Boolean

True/false values:

```
let active:  
boolean = true;
```

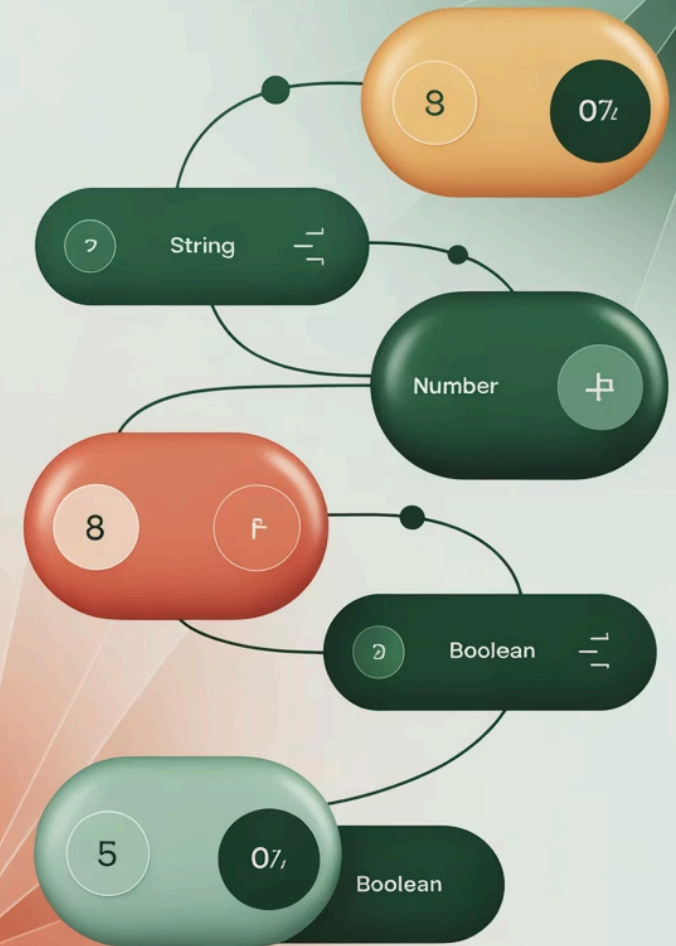
?

Any

Dynamic typing:

```
let value: any =  
"flexible";
```

Typescript Data Types



Type Annotations and Inference

Explicit Type Annotation

```
let count: number = 5;
```

```
let name: string = "Alice";
```

Developer manually specifies the variable type.

Type Inference

```
let count = 5; // inferred as number
```

```
let name = "Alice"; // inferred as string
```

TypeScript automatically determines the type based on the value.

Functions in TypeScript

Basic Function

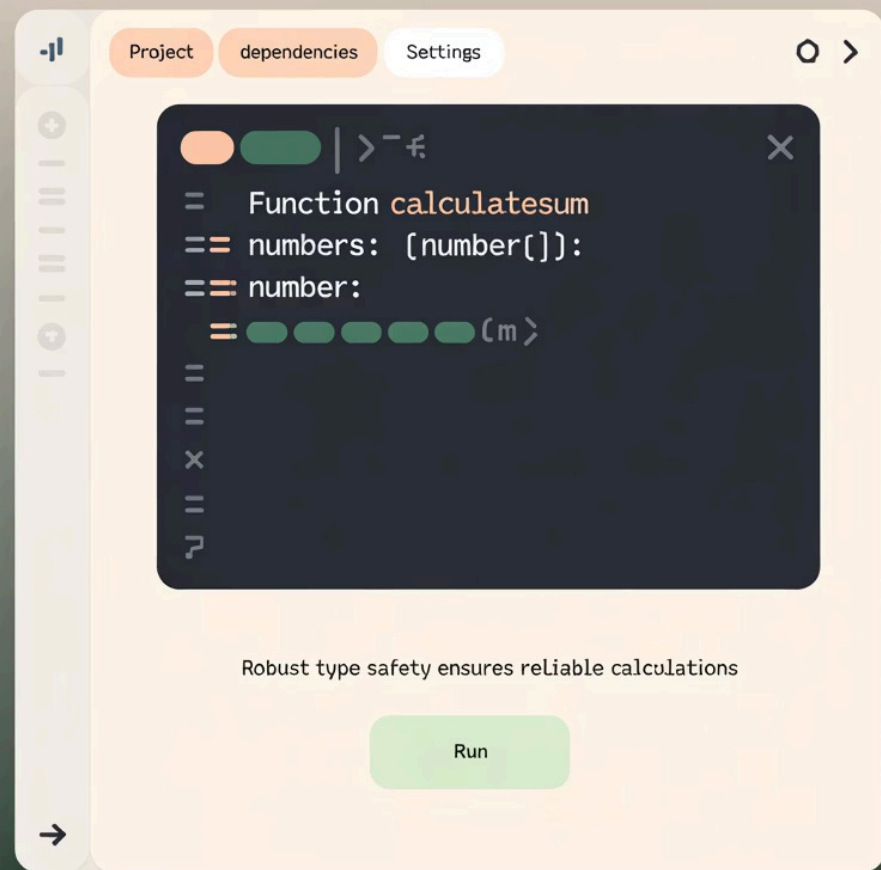
```
function greet(name:
string): string {
  return "Hello " + name;
}
```

Optional Parameters

```
function greet(name?:
string): string {
  return "Hello " + (name
|| "guest");
}
```

Default Parameters

```
function greet(name: string = "guest"): string {
  return "Hello " + name;
}
```



Arrays and Tuples

Arrays

Homogeneous collections:

```
let nums: number[] = [1, 2, 3];
```

Array Methods

TypeScript-aware autocomplete for array operations.



Tuples

Fixed-length arrays with specific types:

```
let pair: [string, number] = ["age", 25];
```

Type Safety

Compiler ensures all operations respect the declared types.

Interfaces



Define Interface

Create object structure contract



Implement Interface

Objects must match the defined shape

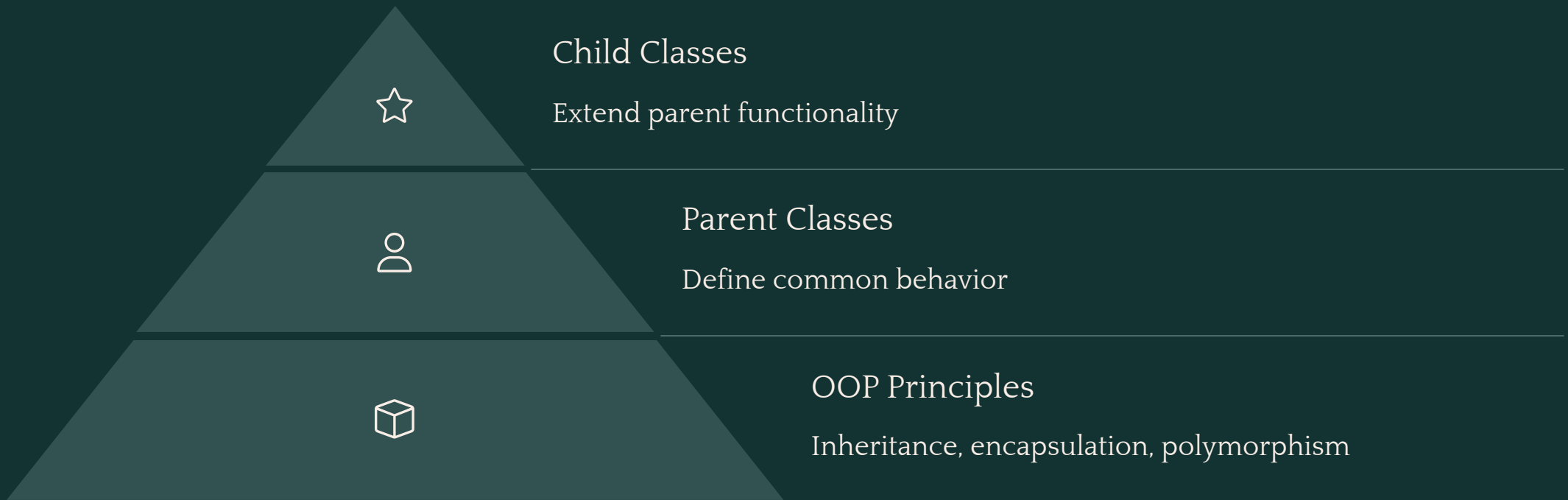


Extend Interfaces

Build complex types from simpler ones

```
interface User {  
  name: string;  
  age: number;  
  email?: string; // Optional property  
}
```

Classes and Inheritance



```
class Animal {  
  name: string;  
  constructor(name: string) {  
    this.name = name;  
  }  
}
```

Enums

Direction Enum

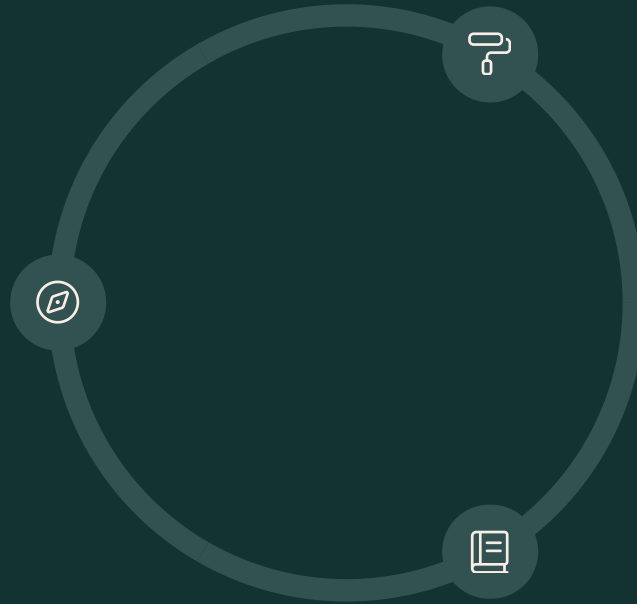
```
enum Direction {  
    Up,  
    Down,  
    Left,  
    Right  
}
```

String Enums

```
enum Color {  
    Red = "RED",  
    Green = "GREEN",  
    Blue = "BLUE"  
}
```

Const Enums

More efficient compiled output for simpler enum use cases.




```
type ID = string | number;
```



```
let id: string | number = 123;
```



```
if (typeof id === "string") { ... }
```



Type

Understand Typescript's power

Explore



Dice perit ouestitheviet los ipetari.

Generics

Generic Functions

```
function identity(arg: T):  
  T {  
    return arg;  
  }
```

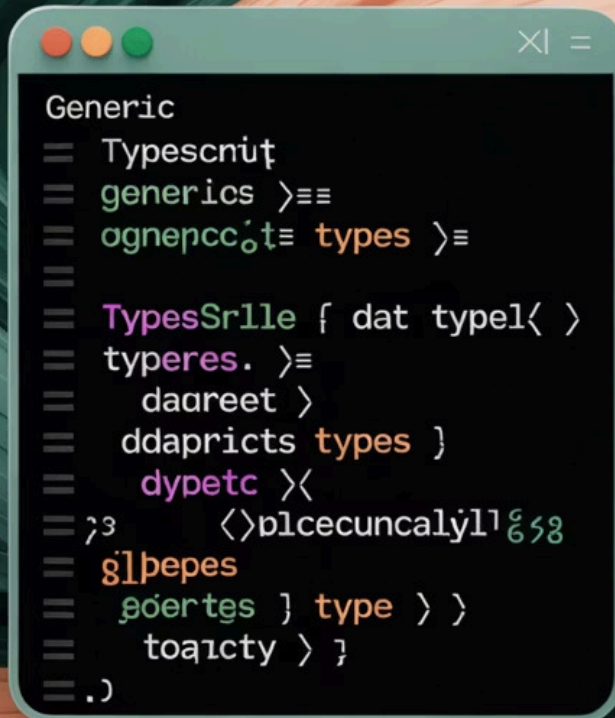
The function works with any type while maintaining type safety.

Generic Classes

```
class Box {  
  value: T;  
  constructor(value: T) {  
    this.value = value;  
  }  
}
```

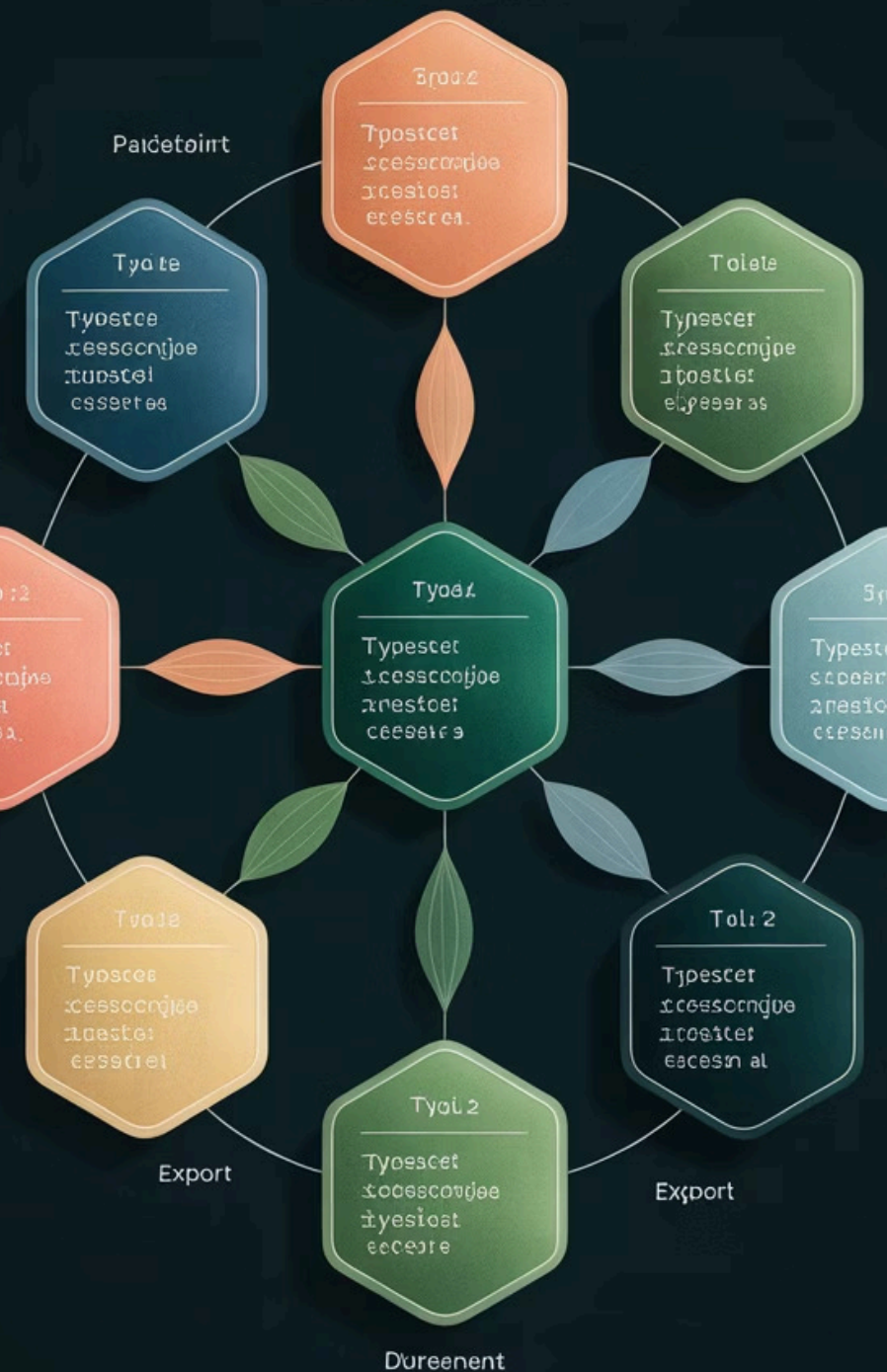
Generic Constraints

```
function getProperty(  
  obj: T, key: K  
): T[K] {  
  return obj[key];  
}
```



TypeScript

Modules



TypeScript Modules

Export

Make items available:



```
export class User { ... }
```

Import

Use exported items:



```
import { User } from './user';
```

Default Export

Main module export:



```
export default class { ... }
```

Namespace Import

Import all exports:

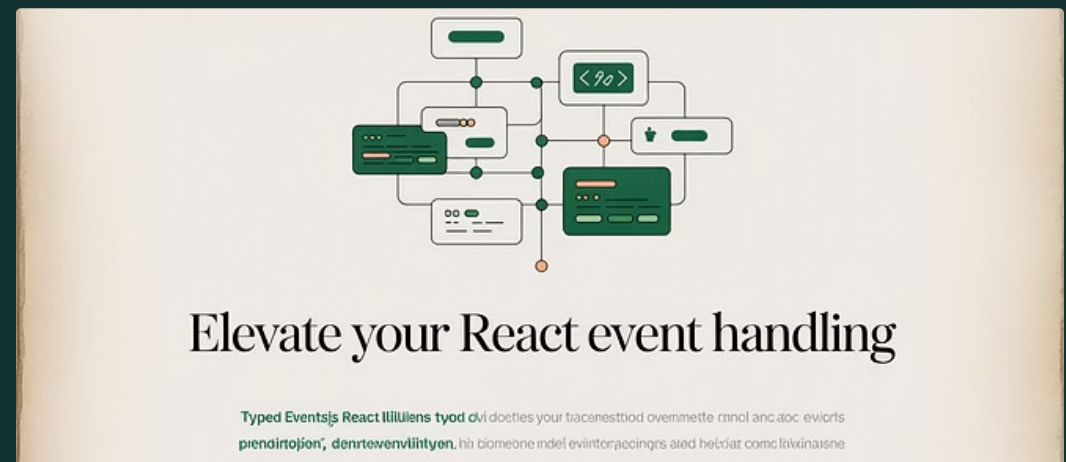
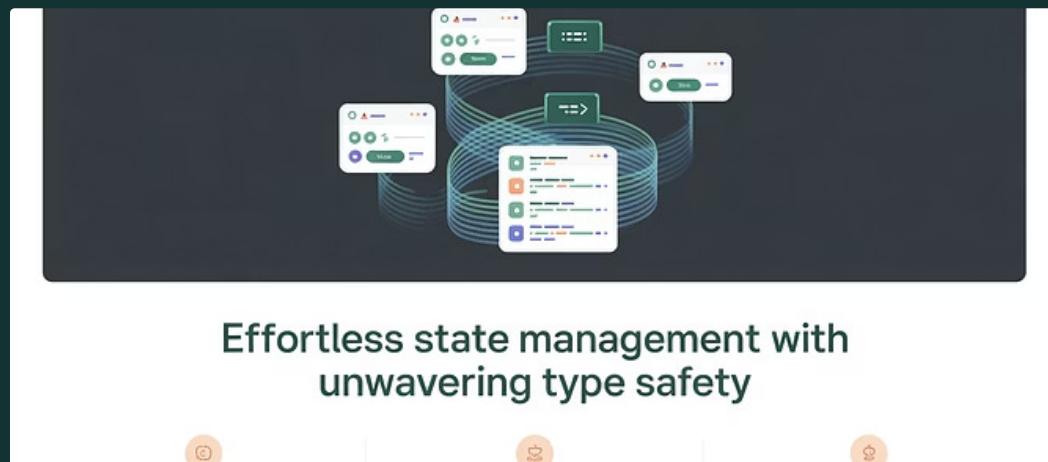
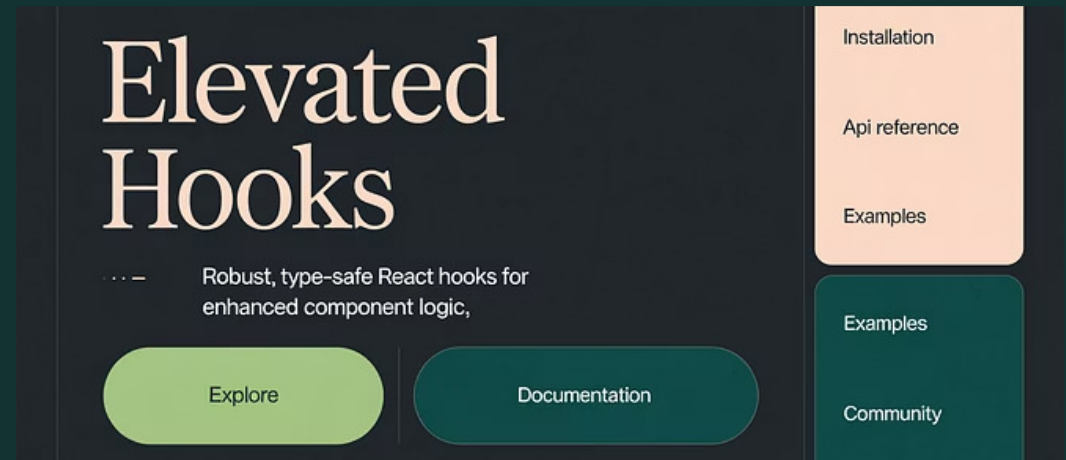
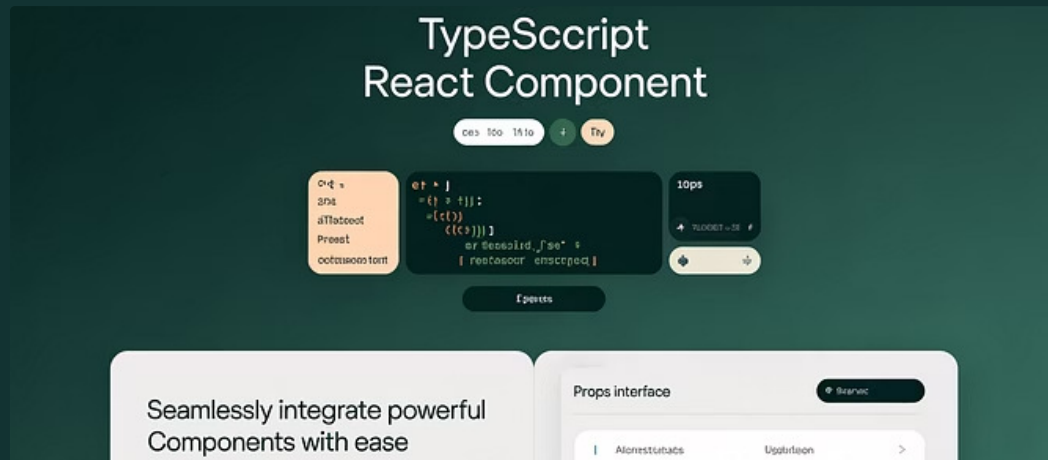


```
import * as Utils from './utils';
```

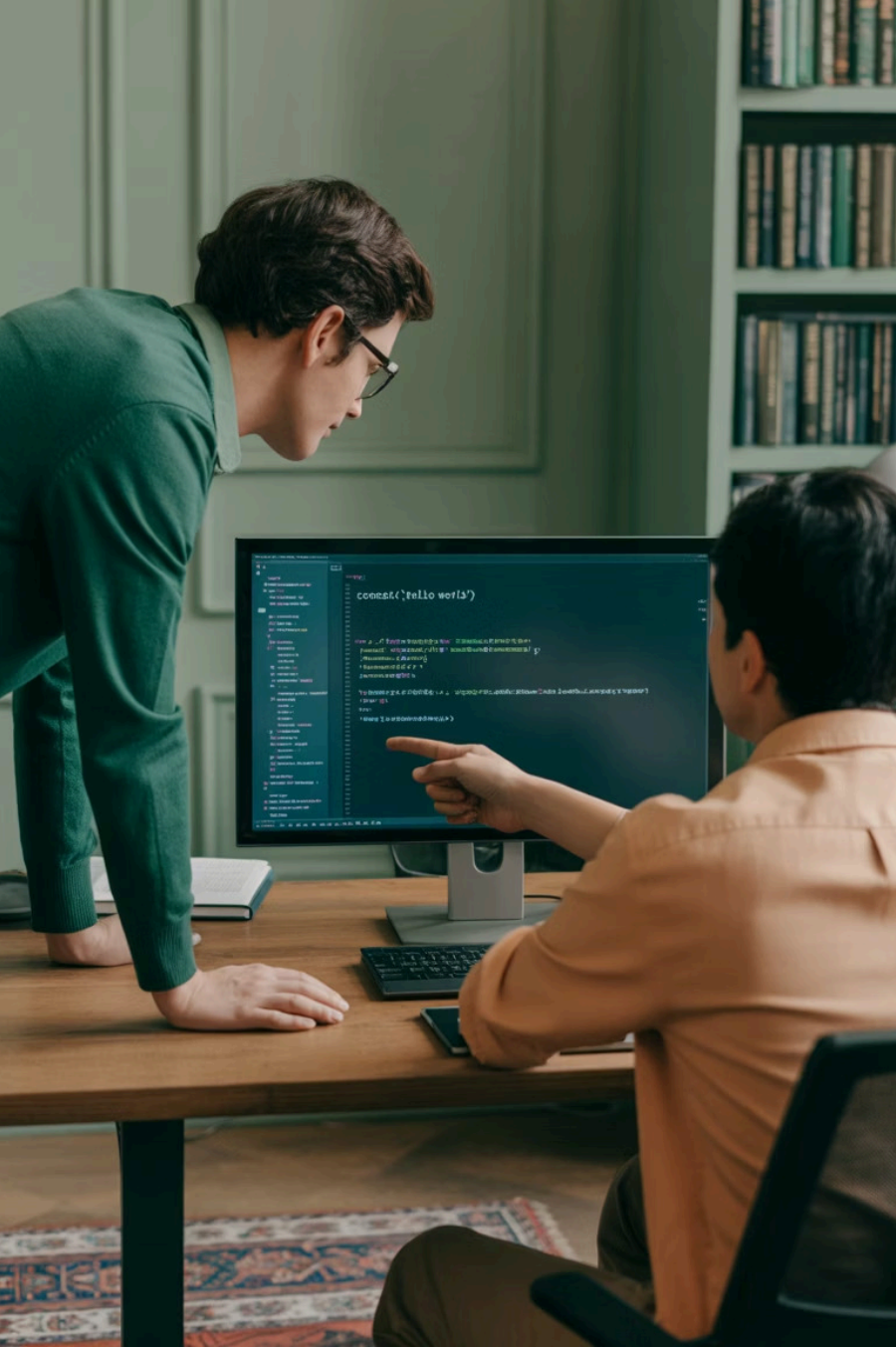
TypeScript Configuration (tsconfig.json)

Option	Description	Example
target	JavaScript version to compile to	"ES2020"
module	Module system to use	"CommonJS"
strict	Enable all strict type checking	true
outDir	Output directory for compiled files	"./dist"

Real-World Example: React with TypeScript



TypeScript enhances React development with type-safe props, state, and events. This improves code quality and developer experience in large applications.



Best Practices in TypeScript



Use Type Inference

Let TypeScript infer types when they're obvious from context.



Prefer Interfaces

Use interfaces for object shapes to improve code readability.



Enable Strict Mode

Use strict type-checking for maximum safety benefits.



Document with JSDoc

Add JSDoc comments for better IDE hints and documentation.

Summary and Resources

2012

First Release

TypeScript has matured into an industry standard.

4.9+

Current Version

Regular updates add new features and improvements.

78%

Developer Usage

Among professional JavaScript developers in 2023.

Start your TypeScript journey at typescriptlang.org with interactive tutorials. The vibrant community offers extensive documentation and support.

